

2025.05.21



LSB隐写技术与分析方法

主讲人：



目录

01 背景与原理

02 设计与实现

03 总结

背景与原理

- 1.任务背景
- 2.LSB隐写原理
- 3.代码分析

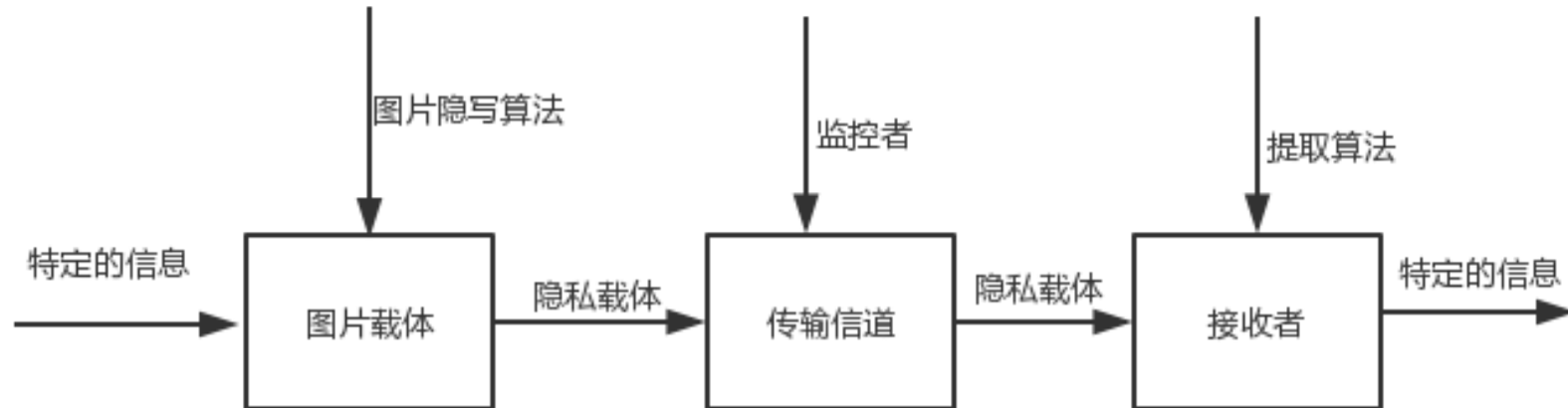


1 任务背景



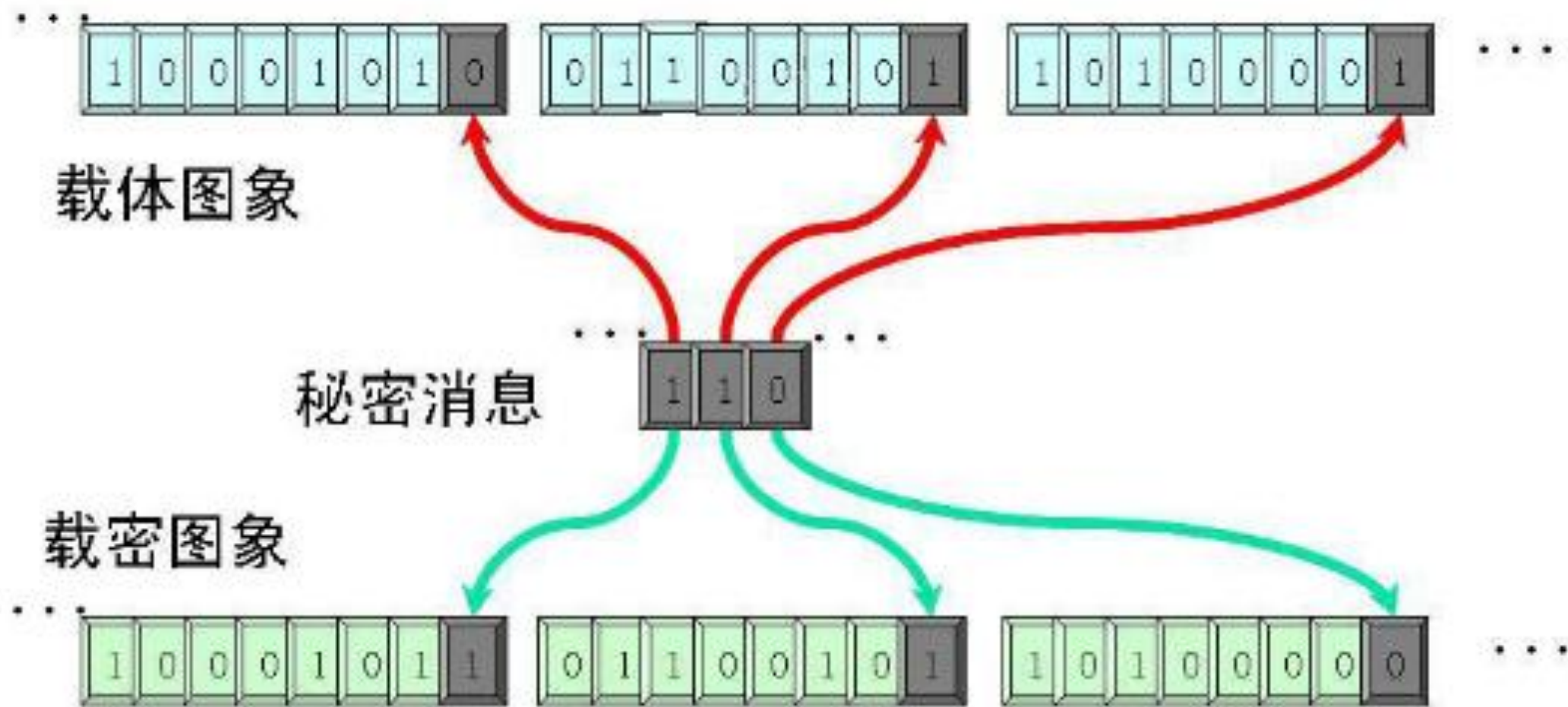
随着互联网的急速发展，人们在通过网络进行数字信息传送和共享的同时，也存在很多安全隐患，如信息的破坏、篡改、丢失等问题也引起各方的重视。

隐写术是信息安全领域的一个重要分支，该技术通常以图像、文本、视频等数字产品为载体，根据载体的特征将秘密信息嵌入到载体中，达到秘密传递信息的目的，有效地保障了秘密信息安全性。



LSB信息隐藏是最早提出的图像空域信息隐藏算法，是将图像**最低比特位的值直接替换为秘密信息**的一种算法。

基本思想



将图像所有像素的不同比特位抽取出来就构成了8个不同的位平面

最低比特位所携带的信息比较少，基本上是噪声

所以不太会影响得到整体图像的呈现，并且用肉眼不会看得出极小的细微差距



标准灰度图像



(a) 第1位 (最高)



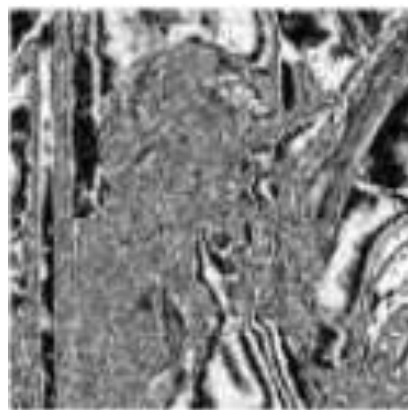
(b) 第2位



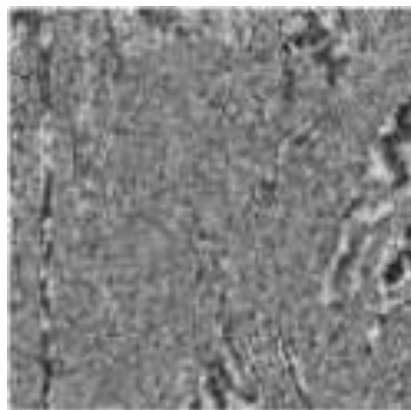
(c) 第3位



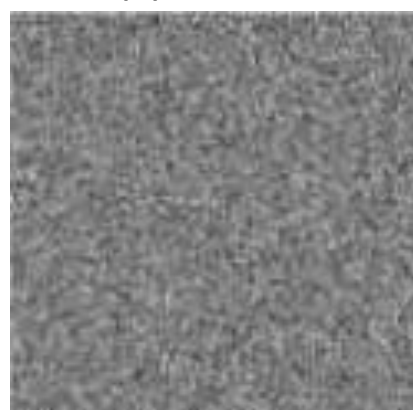
(d) 第4位



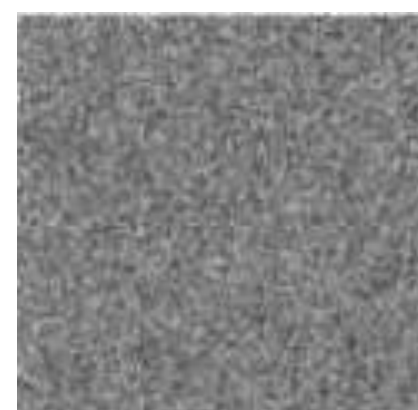
(e) 第5位



(f) 第6位



(g) 第7位

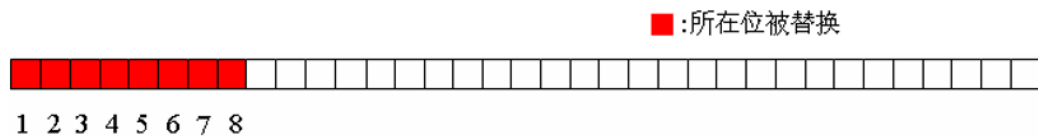


(h) 第8位 (最低)

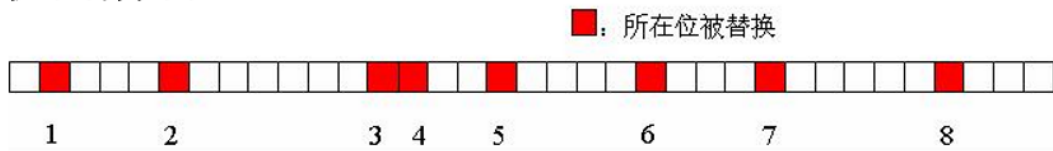


嵌入位置选取方法：连续嵌入法和随机间隔法

连续嵌入法



随机间隔法



连续嵌入法：

按像素顺序嵌入消息，实现简单，效率高，但统计特征明显，易于被检测，

随机嵌入法：

随机选择嵌入消息的像素位置，隐蔽性更高，但需要额外存储位置信息，效率低。

本实验我们使用random随机选择像素点进行嵌入

设计与实现

- 1.LSB信息隐写
- 2.LSB信息提取
- 3.LSB隐写结果



1 LSB信息隐写



1 先读取秘密文件并转换为比特流,

```
with open(secret_file_path, 'r') as file:
    secret_data = file.read()
    binary_secret = ''.join(format(ord(char), '08b') for
                              char in secret_data)
```

2 打开图片按获取像素后嵌入秘密信息

连续嵌入法

按顺序获取图片像素, 按顺序嵌入

```
im = Image.open(image_path)
width, height = im.size
count = 0 # 用于跟踪已嵌入的比特数
for h in range(height):
    for w in range(width):
        pixel = im.getpixel((w, h))
        if count < secret_length:
            pixel = (pixel & ~1) | int(secret_data[count])
            count += 1
        im.putpixel((w, h), pixel)
    if count >= secret_length:
        break
im.save(output_path, format="BMP")
```

随机嵌入法

获取图片像素列表后使用random打乱后嵌入

```
pixels_list = [(w, h) for h in range(height) for w in
                range(width)]
random.shuffle(pixels_list) # 随机打乱像素坐标列表

while count < secret_length and pixels_list:
    w, h = pixels_list.pop() # 获取随机的像素坐标
    pixel = im.getpixel((w, h))
    if count < secret_length:
        pixel = (pixel & ~1) | int(secret_data[count])
        count += 1
    im.putpixel((w, h), pixel)

im.save(output_path, format="BMP")
```



```
def extract_lsb_steganography(image_path,
secret_length):
    im = Image.open(image_path)
    width, height = im.size
    secret_data = [] # 用于存储提取的秘密信息
    count = 0 # 用于跟踪已提取的比特数
    for h in range(height):
        for w in range(width):
            pixel = im.getpixel((w, h))
            if count < secret_length:
                secret_data.append(pixel & 1)
                count += 1
            if count >= secret_length:
                break
        if count >= secret_length:
            break
    secret_message = "".join(str(bit) for bit in
secret_data)
    return secret_message
```

我们这里提取连续嵌入隐写的信息，因为随机嵌入隐写信息一般很难提取出来，除非记录下他隐写的像素位置。

遍历像素值提取每个像素的LSB

提取到指定长度后终止

将比特流转换为字符串输出

选择秘密信息来源（1.随机生成，2.文本文件）：2

256 256

提取的秘密信息（二进制）：01010100011010000110100101110011001000000110100101110
0110010000001100001001000000111001101100101011000110111001001100101011101
0000101100001100010011001000110011001101000011010100110110001101110011100
0001110010110000101100010011000110110010001100101011001100110101001101000
0110100101100111011010110110001101101011011100110111101110000011100010
11100100111001101110100011101010111011001110111011110000111100101111010

提取的秘密信息（文本）：This is a secret,123456789abcdefghijklmnpqrstuvwxy

提取的秘密信息已保存到文件：extract.txt

3 LSB隐写分析



1 肉眼观察



隐写图像的峰值信噪比 (PSNR) 为: 51.16 dB
原始图像的香农熵为: 7.2338
隐写图像的香农熵为: 7.2354



隐写图像的峰值信噪比 (PSNR) 为: 55.32 dB
原始图像的香农熵为: 7.0097
隐写图像的香农熵为: 7.0161

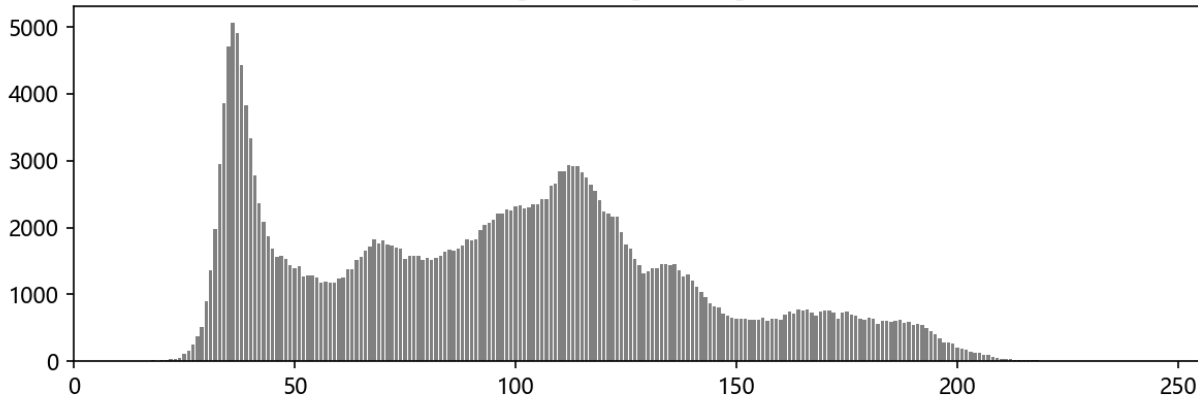
1 LSB隐写分析



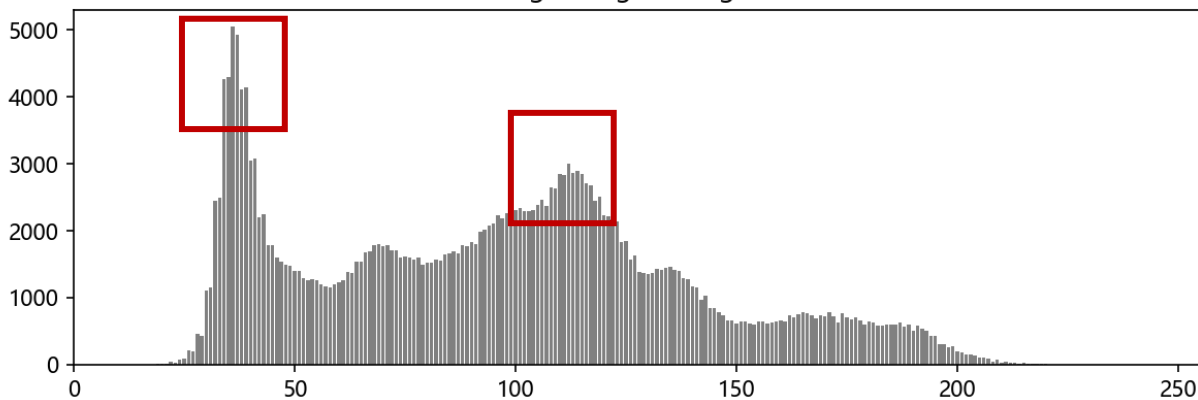
2 卡方分析

是一种用于检验观察频数与理论频数之间差异的统计方法

Original Image Histogram

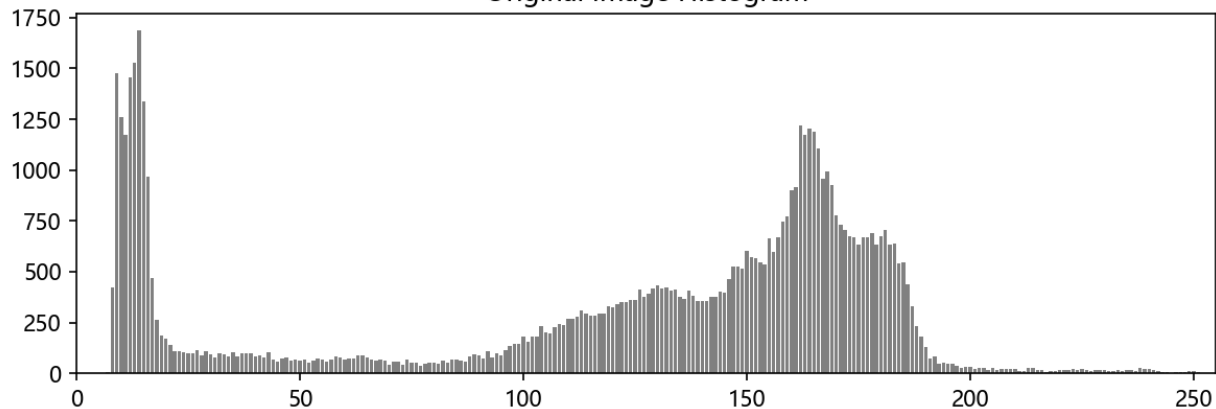


Stego Image Histogram

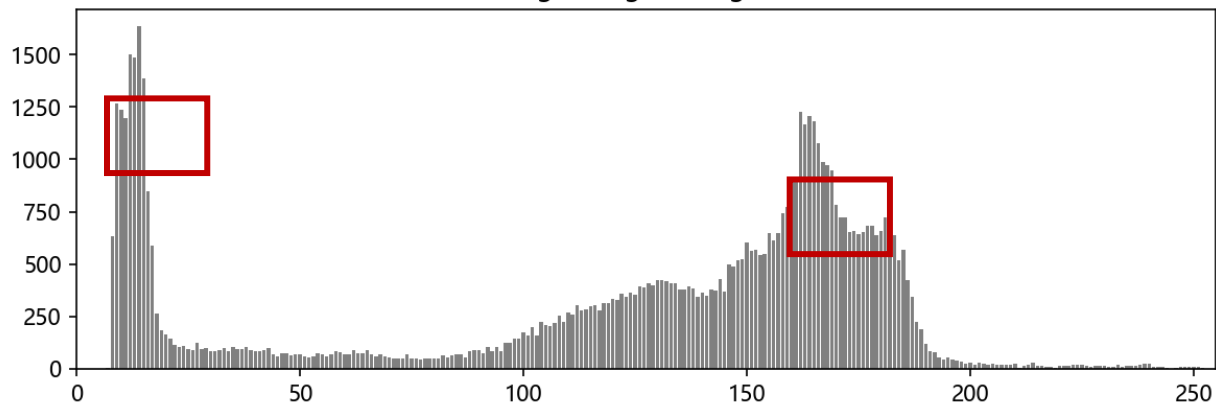


原始图像的r值: 337.6170435673783, P值: 0.0
隐写图像的r值: 46.7733189844239, P值:
0.9999999951625863

Original Image Histogram



Stego Image Histogram



原始图像的r值: 501.5247491861817, P值: 0.0
隐写图像的r值: 227.9912531204539, P值:
2.7683475845030614e-08



信息量估计法

计算直方图的**偶数和奇数位置的像素值的差异**来估计信息的嵌入，计算**f1和f2**，可以量化这些差异，然后通过二次隐写进行检测。

检测时，计算灰度直方图，定义：

$$F_1 = \sum_i |h_{2i+1} - h_{2i}|$$

$$F_2 = \sum_i |h_{2i+2} - h_{2i+1}|$$

如果图象没有经过密写， F_1 和 F_2 在统计上并无不同，因此这两个参数的值应该很接近

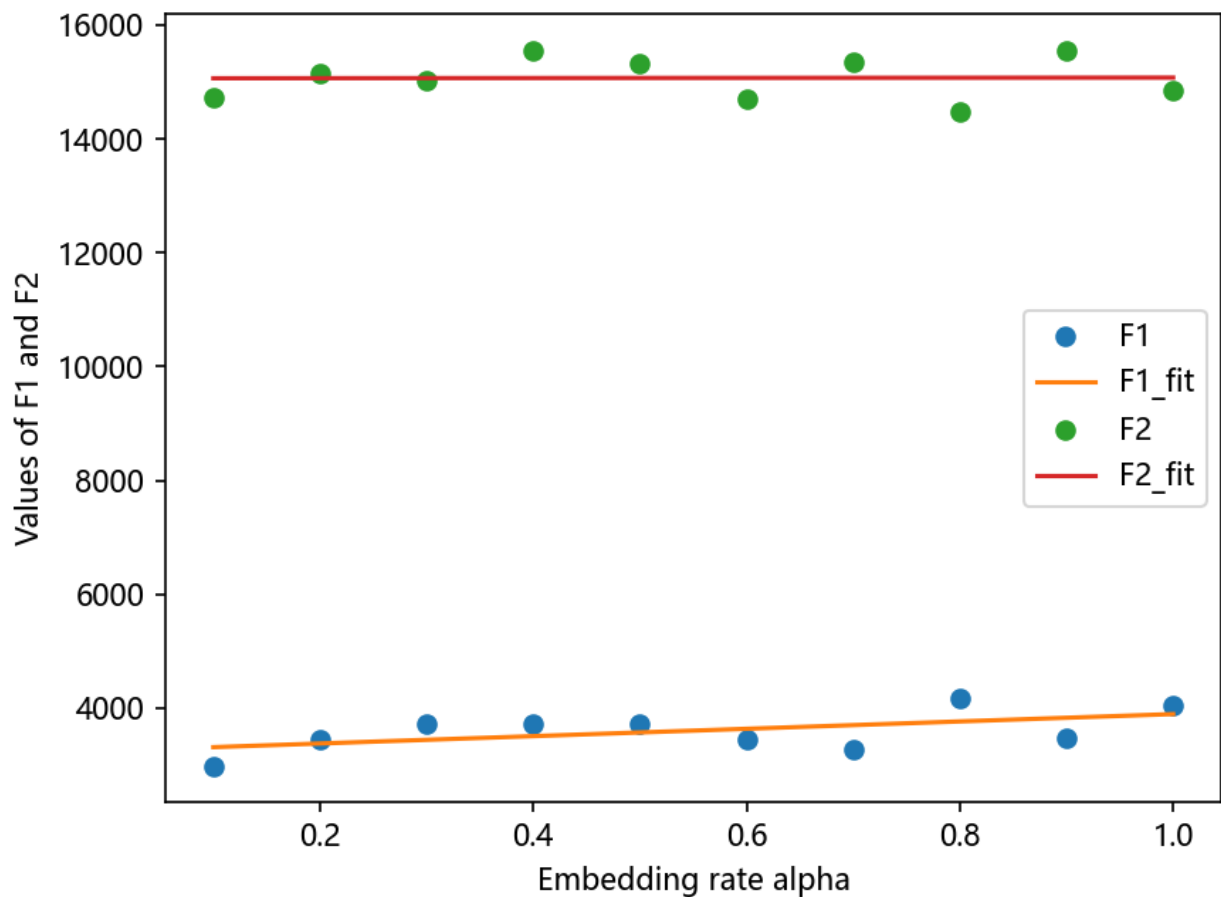
如果图象经过密写， F_1 、 F_2 与嵌入率近似呈线性关系，并且 F_1 随嵌入率增大而减小；而 F_2 不一定减小，即使下降，其速度也远小于 F_1 减小的速度。

如果 F_1 和 F_2 比较接近，可以认为图象中不含秘密

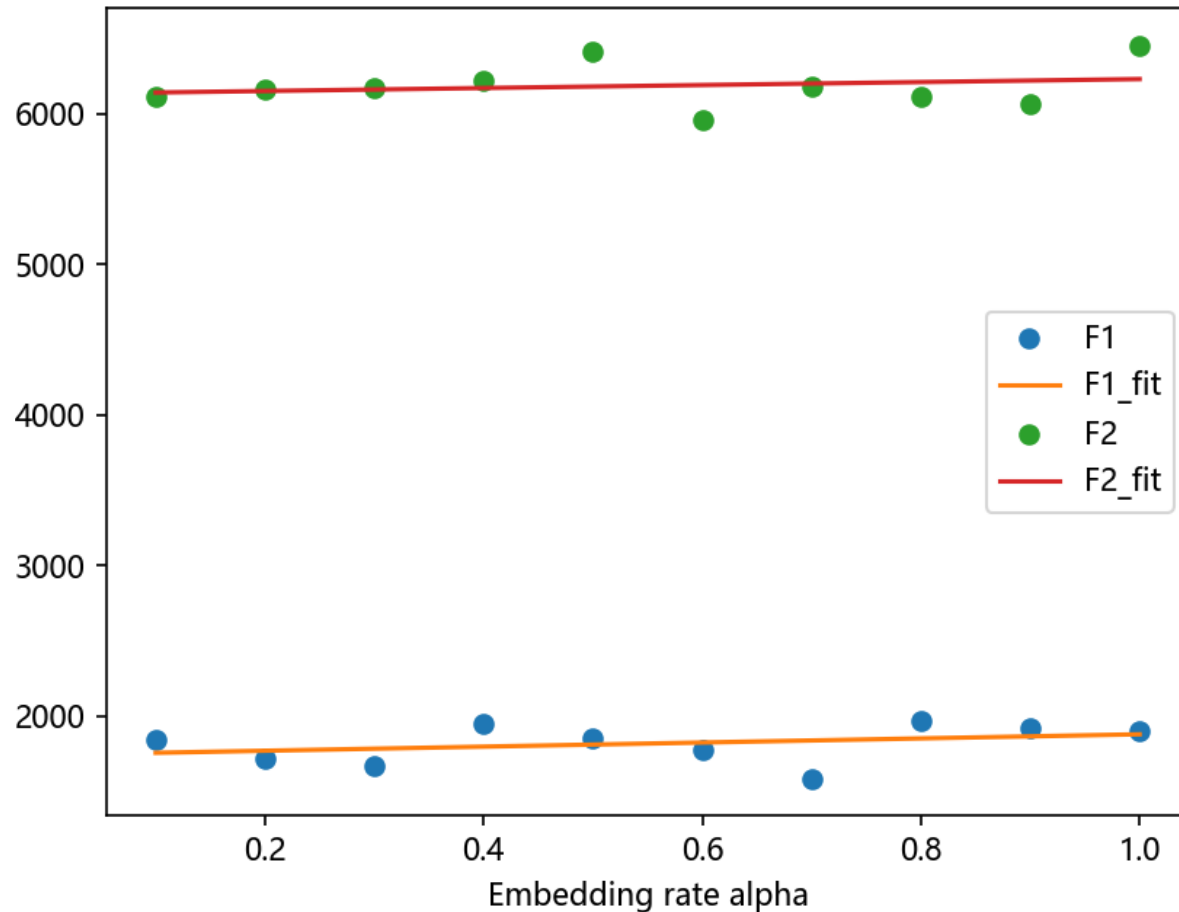
如果 F_2 明显大于 F_1 ，则认为图象中含有秘密信息

信息量估计法

计算直方图的**偶数和奇数位置的像素值的差异**来估计信息的嵌入，
计算**f1和f2**，可以量化这些差异，然后通过二次隐写进行检测。



F1=2784, F2=15002



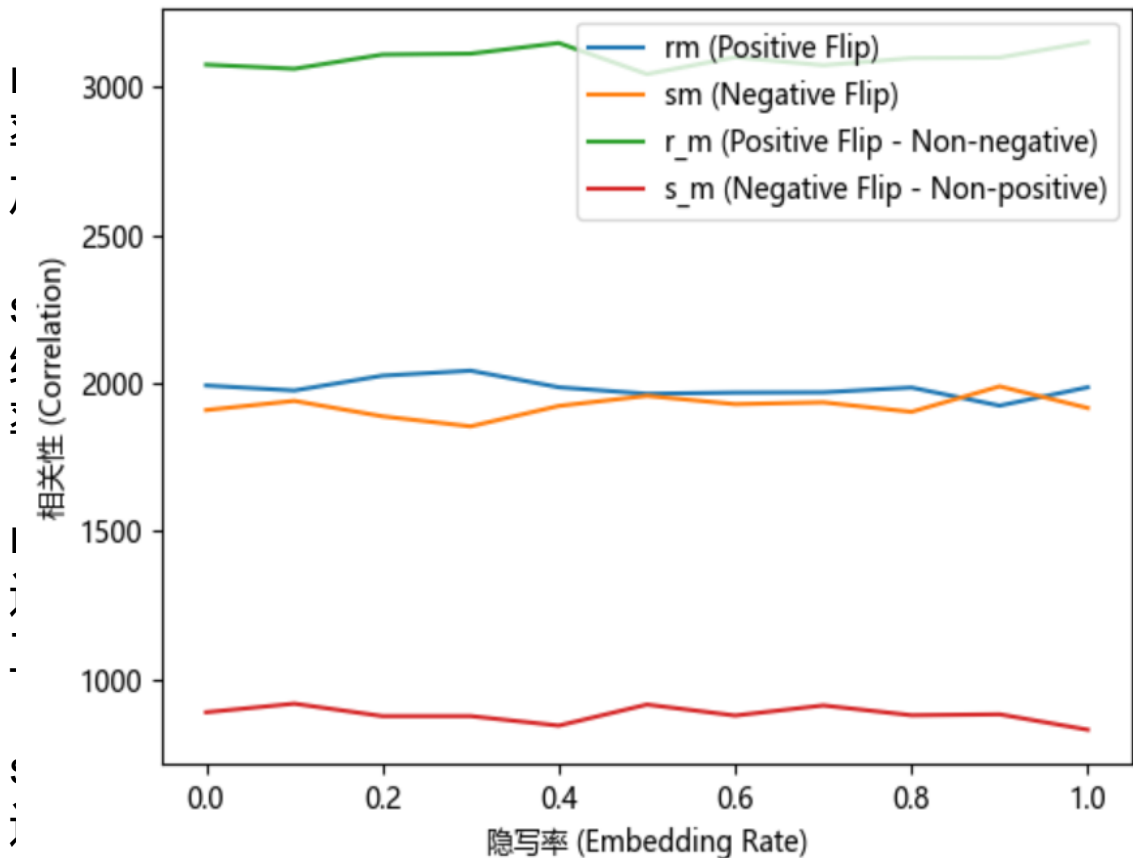
F1=1660, F2=6010



RS分析

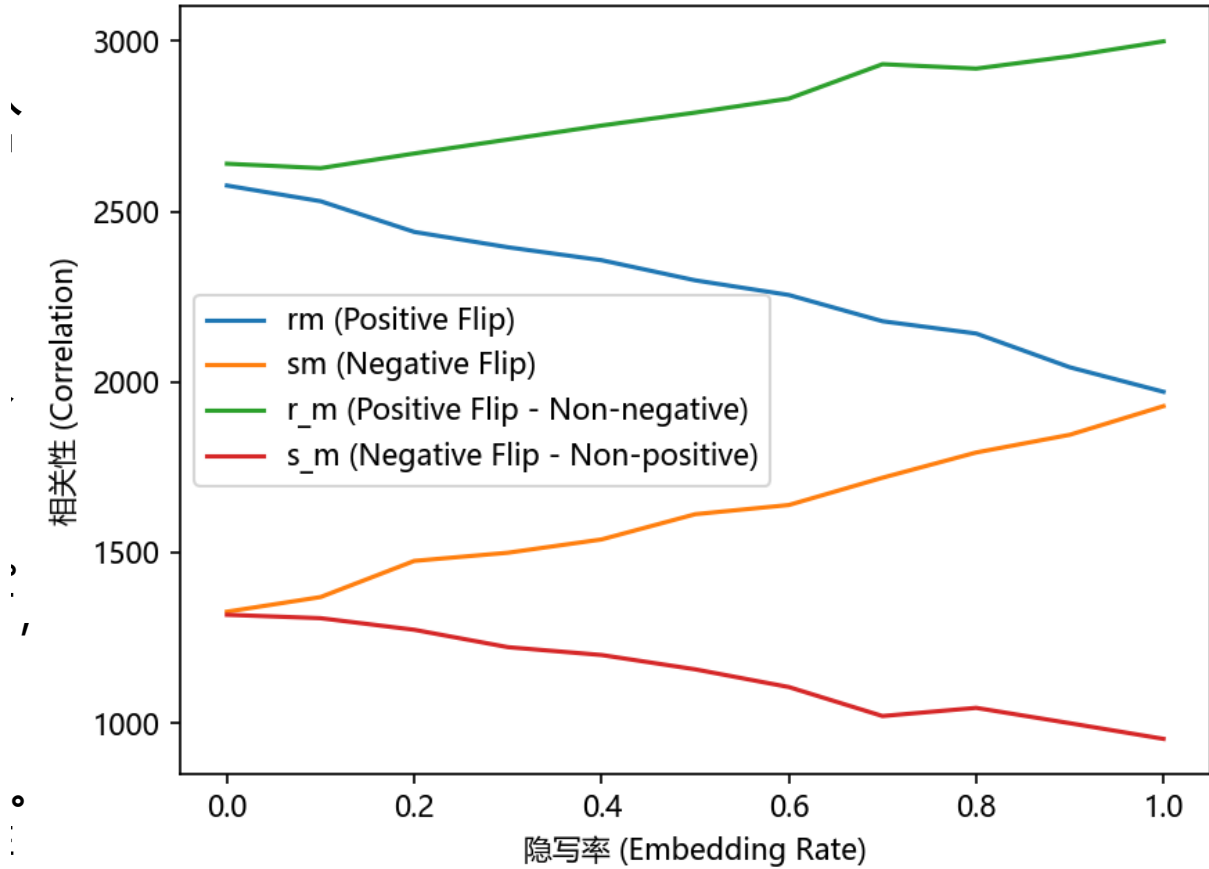
RS分析的核心在于通过**频域统计**来检测隐写信息。

LSB隐写分析结果 (LSB Steganography Analysis Results)



正向特性在高嵌入率下被削弱

LSB隐写分析结果 (LSB Steganography Analysis Results)



总结

1.总结与收获



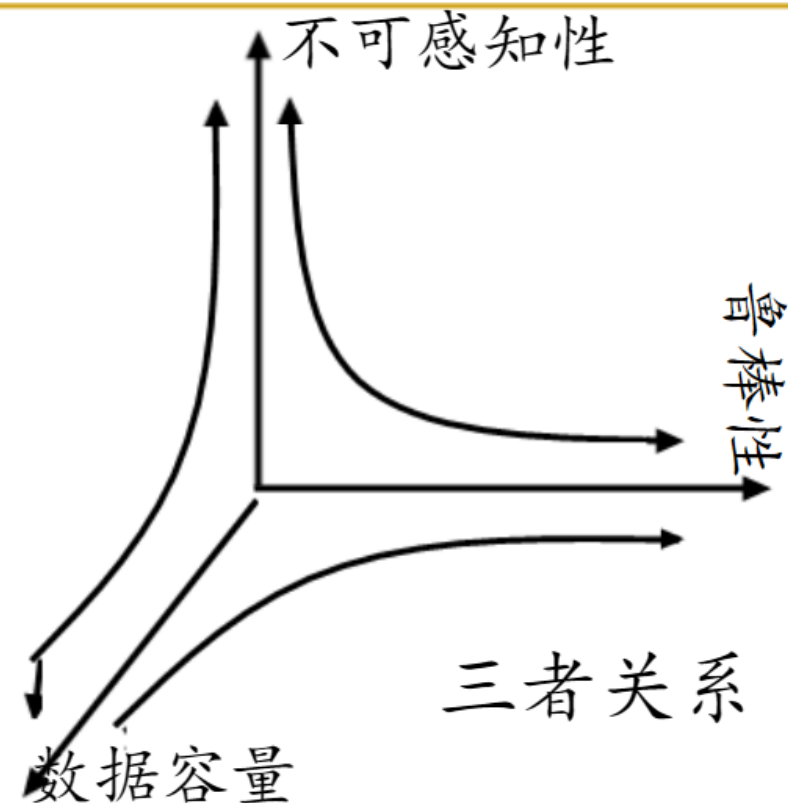
LSB隐写算法过程：

1. 先获取载体图像像素值，将其转为二进制
2. 把秘密信息也转为二进制形式
3. 按顺序逐位替换像素值的最低有效位，完成信息嵌入，因对像素值改动微小，人眼难察觉图像变化。

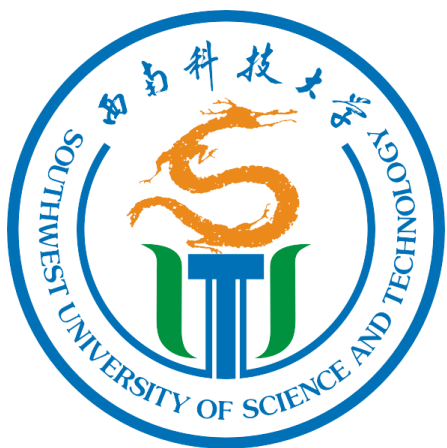
LSB隐写算法简单、隐藏量大，但鲁棒性较差。在隐藏大量信息的情况下依然保持良好的视觉隐蔽性，但使用有效的统计分析工具就可以判断一幅载体图象中是否含有秘密信息。

在信息隐藏领域不可感知性、鲁棒性和数据容量存在一定的关系关系。

在LSB隐写中，不可感知性指隐写后载体无明显变化，鲁棒性是抵抗干扰能力，数据容量为可嵌入信息量。一般而言，不可感知性与鲁棒性、数据容量常呈负相关，需在三者间权衡以达较好隐写效果。



2025.05



谢谢
